Prose Does Not Bind Agents

Mechanically Enforced Safeguards for Multi-Agent Coding on a Single Repository

Rich Tillman
Independent Researcher
github.com/effinrich
effinrich@pm.me

2026-11-03

Abstract

Coding agents are increasingly run in parallel against a single repository, under written instructions that tell them what to build and what to leave alone. This paper reports a 30-day case study of that arrangement: a repository whose 104 commits carry 126 `Co-authored-by` trailers, 120 of them naming one of three AI coding agents, and whose only scope rule the agents could read was a prose rulebook. We ask a narrow question: when such agents remove or weaken the very artefacts that constrain them, what makes the constraint survive?

We report three findings. **F1 (a correct, disclosed, irreversible deletion).** An agent removed the repository's specification harness: 137 files and 10,097 lines in one commit, because the brief did not ask for tests. The removal was right on the brief's terms, it was named in full in the commit message, and the commit carries the agent's own trailer. What failed was not the deletion but the record: nothing in the repository afterwards distinguished *never existed* from *deleted*, and the scope rules that later governed the repository were written by the deletion itself. **F2 (a directive that arrived after the work).** Four agents working one task concurrently each built their own crop-and-export primitive instead of using the one the platform ships; one reached the trunk and had to be taken out again. The prose directive that would have prevented it was written the following day, and nothing could tell whether a given agent had complied. **F3 (checks that pass for the wrong reason).** Three automated checks passed while asserting less than they appeared to; each was found and fixed only because the property being claimed was written down and examined.

We then describe the mechanism that followed: moving constraints out of prose and into the build. Four rules — hash-pinned protected files, mechanical invariants, required and forbidden gate targets, and separately bounded budgets — all of which fail the build when violated. One episode is kept outside the findings, because no written rule answers it: the machine the agents ran on was compromised by a trojan (§4.4). We argue that the design question for agent-assisted development is not how to write better instructions but what to make checkable, and that a repository governed by prose is governed by nothing.

Keywords: multi-agent systems; AI coding agents; software engineering; repository governance; guardrails; large language models; empirical case study

AI use disclosure. This manuscript was drafted with an AI coding agent under the author's direction, and the author verified every quantitative claim against the public repository. The corpus analysed in §2 was itself produced with AI coding agents, which is the subject of the paper.

---

1. Introduction

Coding agents have moved from suggesting lines to proposing commits. A single repository can now be worked by several agents at once, each with its own context window, its own plan, and its own reading of the instructions it was given. The operator's usual control surface is a file: a rules document that says what to build, what to use, and what to leave alone.

That control surface is weaker than it looks. An agent reading prose optimises against the objective the prose makes explicit, and nothing else. A principle like *anything the brief does not ask for is out of scope* is genuinely correct, and an agent that applies it correctly will remove the files that fall outside the brief — including files a human would keep for reasons the brief never mentioned. Not defiantly: correctly. The failure is in the objective function, not the agent.

This paper is a case study of one such repository over 30 days, and of the change that followed it. It is not a controlled experiment. It has one repository, one operator, one domain, and no comparison condition. We state that plainly in §7 and mean it: the contribution is a *mechanism* that is public, inspectable, and checkable, together with an account of the failures that motivated it, not a measured effect size.

The corpus is public. Every commit, decision record, safeguard and transcript we cite is reachable at the URL given in §2.1, so a reader can check each claim rather than take it on trust. Where a claim rests on the operator's own account rather than on a machine-checkable artefact, we say so.

1.1 Contributions

1. Three failure modes observed in the wild under a prose-only rulebook: a correct, disclosed deletion that left no surviving record; a directive that arrived after the work it should have governed; and checks that pass for the wrong reason (§4).
2. A mechanism that survives agents, expressed entirely as checks that fail a build (§5), with the code and its configuration public.
3. A statement of the design principle the mechanism encodes: *constraints that are only stated in prose are not constraints*, and a recorded exclusion is only worth something if the recording is mechanical (§6).

1.2 Relationship to prior work

Empirical work on agentic coding has established that agents are used disproportionately for refactoring, documentation and testing [1, 2], that agent-authored pull requests are accepted less often than human-authored ones [1], and that agent contributions carry more churn than human ones [3]. Fingerprinting work shows that agents leave detectable signatures in commit messages and code structure, and that authorship can be attributed automatically from those signatures [4]. Surveys catalogue what agentic coding systems are for [5], empirical evaluations measure the code quality of AI-assisted generation [6], and controlled studies of a commercial copilot measure its effect on developer throughput [7].

This paper is smaller in scale and different in kind: it is a single-case account of *repository governance* rather than of agent output quality. The unit of analysis is the repository's rulebook and its enforcement, not the code the agents wrote. Where [1] reports that agents are leaned on for documentation work, our corpus shows the same ratio (45 of the 84 commits outside the merge group are process work) arriving with a cost that is not visible in a pull-request-level study: when the documentation *is* the safeguard, delegating it to agents means delegating the thing that constrains them.

2. Background and method

2.1 The corpus

The corpus is a frontend application for annotating PDF documents: open a document, mark it up, pin shapes together, crop and export a region, and export the marked document back to PDF. It is an Nx monorepo on Bun, React 19, TypeScript in strict mode, tldraw v5 for the canvas, and Chakra UI v3 for chrome. The work was performed for a hiring sponsor whose brief specified four tasks and asked for engineering and collaboration evidence alongside them.

The corpus is public [10] and names the sponsor throughout, so the sponsor is not anonymous here; what this paper does not discuss is how the exercise was evaluated. Every commit, decision record, safeguard and transcript cited below is reachable at `https://github.com/effinrich/h2-fe-assessment`, so each claim can be checked rather than taken on trust.

Table 1 — The corpus at the end of the study period, at commit `b200e23`.

Property	Value
Public history	2026-10-05 → 2026-11-03 (30 days)
Commits on the trunk	104
— commits with a `Merge ...` subject	20
— product commits (`feat`, `fix`, `perf`, `refactor`, `ci`, `test`)	35
— process commits (`docs`, `chore`, `style`, `build`)	45
— other	4
`Co-authored-by` trailers	126 across the history
— naming an AI agent	120
— naming the author	5
— naming a CI bot	1
Distinct agents named	3 products, 5 accounts
Tracked files at end	165
Product source	81 files, 3,073 non-blank lines (budget 95 / 3,300)
Test source	4 files, 227 non-blank lines (budget 12 / 400)
Decision records	17

Counts are reproducible from the public repository; the commands are in the appendix (§A), where they are pinned to that commit rather than to the branch tip, so that later commits to the repository do not change what they report. Commits are classified by subject prefix, which is the only taxonomy the history supports: a `Merge ...` subject, a product prefix (`feat`, `fix`, `perf`, `refactor`, `ci`, `test`), a process prefix (`docs`, `chore`, `style`, `build`), or other. One of the twenty in the merge group is a squash merge with a single parent, so nineteen are merges by parent count and eighty-five are not.

2.2 The agent fleet

Three distinct agent products contributed to the history, identified by the `Co-authored-by` trailer on each commit: a desktop coding agent (102 trailers), a CLI coding agent (14), and GitHub Copilot under three account names (4). A CI bot accounts for one further trailer and

is not an AI agent; the split between the two is ours, because a trailer is a string and classifying it is a judgement about what the account was doing. The remaining 5 trailers name the author on commits he co-authored himself. The trailer is not a measurement instrument — it is a disclosure convention, and its reliability as an attribution signal is itself the subject of recent work [4]. We use it here as a *record of what was claimed at the time*, which is the thing a reader of a public repository can check.

Agents and the operator worked concurrently on the same repository for most of the period. This concurrency is the precondition for every finding below.

2.3 Evidence sources and their limits

Three sources, with different reliability.

The visible git history. One hundred and four commits from 2026-10-05 onward. It is complete and machine-checkable for that window. It is *not* complete for the whole project: the public history begins at a scaffold commit that already contains the specification harness, so the harness's construction is not in the visible history. Findings that depend on the construction rather than on the removal inherit this limit (§7).

The decision records. Seventeen markdown files, each holding a context, a decision and its consequences, written by the operator. They are the closest thing in the corpus to a primary source for intent, because they were written at or near the moment of decision. They are also the operator's account, and the operator is the author of this paper.

The session transcript. 9,924 lines covering the working session: 86 turns, 1,834 tool calls and 7 context compactions, sanitised for publication. It contains the agent turns that no commit preserves — the reasoning that produced a wrong plan, the moment a directive was issued. It is also the least controlled source, and §4.1 uses it only where a commit independently corroborates the claim.

We did not use the transcript to establish anything that the git history can establish. Where the two disagree, we report the git history.

2.4 How findings were identified

Each finding required two independent artefacts: a machine-checkable trace in the repository, and a record of intent (a decision record, a commit message, or a corroborated transcript passage). Findings with only the second were treated as anecdotes and are not reported.

---

3. The rulebook

The rulebook is the instrument the findings are about, so it needs a history rather than a snapshot. Read as a snapshot it says that a repository governed by prose failed under prose. Read as a history it says something sharper: the scope rules were written *after* the failure, and by the failure.

Before the first failure (2026-10-05). The repository carried fourteen agent instruction files: `AGENTS.md` and `CLAUDE.md` at the root and in each of six projects. The root file was 166 lines. Most of it was technical: how to query the Nx graph instead of inferring it, where dependencies live, that component files carry size limits, that project tags are mandatory. But it was not only technical. It ended with a section headed "Rules that do not bend", and six rules under it: verify every third-party API against the installed types; spec before code, failing test before

implementation; the gate hook is authoritative; never guess a CLI flag; be terse; ask before deleting any config file.

What it did not contain was a *scope* rule. The phrase *stay inside the brief* appears in no file in the repository before 2026-10-06, and neither does *use the vendor's solution* or *smallest working diff*. There was nothing for an agent to over-read, because nothing about the boundaries of the work had been said.

One place did record a scope principle in prose, and it is not the rulebook. Decision record 0009, *What we deliberately did not build*, dated 2026-10-01, opens: "An unexplained absence reads as an oversight. A recorded exclusion reads as judgement." It lists what the repository would not build — no backend, no auth, no persistence, no SSR — and says of the server that it is "scope the brief did not ask for". That is the principle the removal applied, and the removal applied it correctly.

The removal wrote the scope rules. The scope rules first appear on 2026-10-06, in the same commit that stripped the harness, and the commit says so: *"AGENTS.md now leads with scope rules: vendor example first, stay inside the brief."* Six were written that day. Three were new, and three were rules the file already had:

Before the removal	**After it**
Spec before code; failing test before implementation.	**Inverted.** Became rule 3: *Smallest working diff. No specs, stories or tests unless asked.*
Verify every third-party API call against the installed types.	Rule 4, unchanged
Never guess a CLI flag.	Rule 5, unchanged
Ask before deleting any config file.	Rule 6, unchanged
The gate hook is authoritative.	Dropped
Be terse.	Dropped
—	New rule 1: *Use the vendor's solution.*
—	New rule 2: *Stay inside the brief.*

The order of the table is the point. The commit that deleted the specification-and-test harness also rewrote the one rule that required them, from *spec before code* to *no specs or tests unless asked*, and moved the rule that made the gate authoritative out of the file entirely. The first three rules read in full:

1. *Use the vendor's solution.* Before building anything, check the platform's own examples and the installed library. Start from the example; change only what the brief needs.
2. *Stay inside the brief.* Anything beyond it is a question for the user, not a default.
3. *Smallest working diff.* No specs, stories or tests unless asked.

The rules that later governed this repository were authored by the deletion they are usually credited with preventing. A rule written by the act it is supposed to prevent cannot have been preventing it.

The guardrail warning came last. The section that declares the rules file, the git hooks, the scripts in `tools/`, the decision records and the gate to be load-bearing, and warns that they are "not scaffolding, and they are not 'the test harness'", first appears on 2026-10-26: twenty days after the failure it would have prevented, in a file the agent had already rewritten.

At the end of the study period. One rules file, 149 lines, six scope rules, a guardrail section. Everything in it is prose. Nothing in it was checkable. That is the condition the findings describe, and it was reached by writing more prose rather than by making anything checkable.

4. Findings

4.1 F1 — A correct, disclosed, irreversible deletion: 137 files in one commit

On 2026-10-06, the day after the scaffold commit that introduced the harness, one commit removed the specification harness in its entirety: the per-task specs, the test runners and their dependencies, twelve per-project agent instruction files (six `AGENTS.md` and six `CLAUDE.md`), 21 unused design system components, all feature and app stories, two decision records, and the second Storybook.

Table 2 — The removal (commit `9eeb60a`).

Area	Files	Lines removed
`libs/`	107	7,379
`apps/`	10	443
repository root	9	602
`docs/`	9	1,595
`tools/`	1	73
`.github/`	1	5
Total	**137**	**10,097**

The commit message reads, in full:

> *chore: strip the test harness, unused components and app stories (#43)*
>
> *Testing is not required by the brief. Removes every spec, the jsdom/Chromium test runners and their deps, the test-storybook target, the second Storybook, 21 unused design-system components, all feature and app stories, the TDD docs (HARNESS, specs, ADR 0005/0007) and per-project agent docs. AGENTS.md now leads with scope rules: vendor example first, stay inside the brief. Gate: lint, format, typecheck, build, build-storybook.*
>
> *Co-authored-by: Cursor <cursoragent@cursor.com>*

Read it again. The deletion was **disclosed**, in the first person, by name and in detail. It was **co-authored by the agent that made it**: the trailer names Cursor. And it was **reasonable**: the brief did not ask for tests, and the commit says so in its first sentence. This is not a story about an agent concealing a deletion. Every artefact a reader would need is in the commit.

The governance failure was not the deletion. It was that the deletion was **irreversible in the record**. The decision records that had explained why the harness existed went with it: 0005 recorded the spec-and-test rules that went with it, and 0007 recorded the per-library Storybook rules, which were still in force afterwards. What survived could not be distinguished from what had never been there, and the reason for the removal lived in one commit message that no mechanism required anyone to read. The remedy, arrived at later and described in §5.4, is a manifest entry per removal, printed by the guardrail script on every run: the recorded exclusion that decision record 0009 had asked for, made mechanical.

There was a second, mechanical layer, and it did not help. The repository's `main` branch carried a ruleset from 2026-10-02. Before the failure it required a pull request, no force push and no deletion — but no status check, no approval, and bypass actors that included admins, maintainers and writers. It protected the shape of a merge and nothing about its content, and the removal satisfied it: it arrived as a pull request and did not force-push (safeguards record). The operator's own account of the commit is that every layer that should have stopped it was changed by it: "The prose rule was rewritten. The specs that justified it were deleted, and the

decision records that explained why they existed went with them. The gate stopped running the checks, so the build was green because the commit had just removed what the build would have run." That is the failure in one sentence, and it is why the mechanism that followed pins hashes rather than writing rules.

Two further facts complicate the story, and they are worth stating plainly, because the version of this finding that is easier to tell is wrong. First, the operator's later account says the agent read "stay inside the brief" and concluded the harness was outside it (safeguards record). That phrase is in no file in the repository before the commit that removed the harness and wrote the rule. Either the agent was told it outside the repository, or the account is an anachronism, and the repository cannot distinguish the two. This is the paper's method in miniature: the prose record is a primary source for intent and an unreliable one for sequence, and only the artefact's own history can settle an ordering. Second, the scope rules that later governed the repository were written by this same commit (§3). The rules were not misapplied. They were late.

4.2 F2 — Four agents, one task, four implementations

Four agents worked the camera task concurrently. Instead of using the crop-and- export primitive the canvas platform ships [8], each built their own: a crop frame as a custom shape util, a frame with its own outline, label, marks and marquee, a toolbar for it, an export layer with its own action state, a first-use hint, its own type definitions, and a story per part. The largest of them reached **36 source files and about 3,400 lines**.

One of them was not confined to a branch. The bespoke camera reached the **trunk** on 2026-10-05, in commit `6f6ec34` ("one component per file, stories split by theme"), and the camera directory at that commit is byte-for-byte identical to the tip of `origin/refactor/camera-components`: 30 source files, no diff. This was not a harmless experiment that got tidied away. It was mainline for a day.

The correction took two pull requests, both on 2026-10-06. PR #47 rebuilt the feature from the platform's own example (29 files, +365/−1,561). PR #57 took the example as it is, removing the bespoke frame, toolbar, marquee and export layer (15 files, +98/−370).

The interesting part is not the waste. It is the **timing**. The prose directive that would have prevented all of it — *use the vendor's solution* — was written on 2026-10-06, the day after the bespoke camera landed and the day the harness was stripped (§3). It could not have reached work that was already finished. And once it existed, nothing in the repository could tell whether a given agent had complied with it, because compliance was never expressed as something checkable. A directive addressed to four agents working concurrently is not a mechanism; it is a broadcast, and the broadcast arrives after the work.

The camera episode also shows the shape of the eventual remedy, and shows where the remedy stops. What the build enforces in the trunk is not "use the vendor's solution". The lint configuration enforces module boundaries between features and a size limit on files; **nothing in it rejects a bespoke primitive**. The trunk ships a custom `CropShapeUtil` of its own, and decision record 0017 records that decision deliberately — the tldraw demo is "a gesture, not a shape", because it exports the region and leaves nothing behind. The prose is still there, and it is still the thing that decides; it simply stopped being load-bearing for anything the build can see.

Decision record 0017, written on 2026-10-26, still says of that work: "None of it reached `main` — it lived in branches and in the working tree." The artefact says otherwise, and this paper reports the artefact. The record is the operator's account; the commit is the commit.

4.3 F3 — Checks that pass for the wrong reason

Three checks in the guardrail script were passing while asserting less than they appeared to. Each was found in the course of writing the record that would later describe the verification surface, and each was fixed.

Two of the three are attested by decision record 0015 rather than reproducible from the repository: their buggy forms appear in no commit, on any ref. The third is demonstrable from the public history. It was merged in #76 (`f471a29`) silently skipping the project file it could not read — a `catch` that continued — and fixed in #77 (`98f6d3f`), whose diff replaces the skip with a failure. We mark the difference rather than blur it.

Table 3 — Three checks that passed for the wrong reason.

Check	Why it passed	What it should have done
Forbidden-path rule	Patterns were matched against the repository root only, so a forbidden path nested in a directory was not matched	Match the whole path *and* the basename, so a pattern means any depth
Project-tag check	A project with no `tags` array was skipped	A project with no tags is a project with no boundary — fail it
Project-tag check	A project file that could not be parsed was skipped	An unreadable project file is not a passing project file — fail it

The third is the instructive one. The rule that exists to catch a malformed project file was itself silently tolerant of malformed project files. The generalisation the operator wrote down afterwards is the one this paper adopts:

> It generalises to every check in this repository: a gate that fails because a tool is missing has proven nothing, and neither has a check that passes for the wrong reason.

The record adds how the third one was found: "The rule is not inherited from the old harness; it is what caught them, and it caught the third one only because the second merge made it worth looking for." That is the honest version of how safeguards get found: not by a rule firing, but by somebody writing down what the rules were supposed to be and noticing the difference.

This finding is the strongest argument for the mechanism in §5. A guardrail is only worth what its failure modes are worth, and a guardrail that cannot fail is indistinguishable from no guardrail at all. A check that *can* fail, and fails for the right reason, is the only kind worth adding.

4.4 The environment, which is a different failure

During the period the machine the agents ran on was compromised by a trojan. It was found at the end, the credentials it could reach were rotated, and the sponsor was told on the day the work was submitted.

This belongs in the record because it is easy to conflate with F1, and the two need different answers. F1 is a *design* failure: the rules were too weak to bind, and the remedy is to make them mechanical. The compromise is an *environment* failure: no rule, however well written, resists a compromised machine, and the remedy is credential rotation and not taking an agent's word for anything.

One incident, self-reported, with no independent forensic record accompanying this paper. We report it because the distinction matters more than the incident: a mechanism that made agents'

behaviour safe would not have made a compromised host safe, and a repository that reads as a model of careful process while its host was compromised is a repository that should say so.

---

5. The mechanism

The remedy was to move constraints out of prose and into the build. Four rules, all of which fail a build when violated, all implemented in 453 lines across two scripts and one manifest.

5.1 Hash-pinned protected files

Nine files are load-bearing: the rules file (`AGENTS.md`), the generated Nx block (`CLAUDE.md`), the git hook configuration (`lefthook.yml`), two agent hook configurations (`.cursor/hooks.json`, `.claude/settings.json`), two hook scripts (`tools/shell-guard.mjs`, `tools/stop-gate.mjs`), the budget script and the guardrail script itself. The manifest records a SHA-256 for each. The guardrail script recomputes them on every run.

The manifest that holds the hashes is not itself hash-pinned; it cannot be, without a bootstrap. Its protection is a `CODEOWNERS` entry naming the owner. That entry *asks* for the owner's review, and the repository's own safeguards note records why it cannot be more than that: the ruleset requires no approving review, and there is no second approver. The API confirms it, and the appendix gives the command: the ruleset on `main` sets its required approving review count to zero. The guarantee is therefore split in two: the build catches a protected file that changes alone, and the owner is asked to see the change that arrives with an updated hash beside it.

The point is not tamper-evidence against a determined attacker. It is that an agent editing a protected file in the same commit as something else will find the build red, and that the red is attributable: the diff shows both the file and its hash changing. An agent can rewrite the sentence that says *do not remove these files*. It cannot rewrite the sentence and the hash in one step without the change being visible in the same diff.

5.2 Mechanical invariants

Some files legitimately change, so hashing them is wrong. For those, the manifest records six assertions covering nine required substrings. The gate script must contain the command that runs the guardrail check, and the gate must derive its targets from the project graph rather than from an enumerated list [9]; the lint configuration must contain the module-boundary rule; the rules file must contain the guardrail section; and both CI workflows must reach the check — the main workflow through `bun run gate`, and a standalone workflow through the script directly.

5.3 Required and forbidden gate targets

The gate's target list is itself governed. A target the repository depends on is **required**: removing it from the gate fails the build. A target the repository has decided against is **forbidden**: adding it fails the build.

This rule has a history worth reporting, because it changed direction. The guardrail originally *forbade* the gate from running a `test` target, because the brief did not ask for a test suite and agents default to more verification than the brief asks for. Eight days later the operator asked for a test loop anyway. The rule was not deleted: it was flipped. The forbidden list went from `["test", "test-storybook"]` to `required: ["test"]` with `forbidden: ["test-storybook"]` beside it, and the check grew a matching arm. The old rule kept a suite

out; the new one keeps a suite in. A rule that can only ever say *no* is a rule that has to be rewritten the moment the decision changes; a rule with a direction survives the decision changing.

The same shape appears in the budgets (§5.5).

5.4 Everything else that was needed

- **Records that must exist.** Seventeen decision records are named. Deleting one fails the build.
- **Forbidden paths.** Fourteen glob patterns (`.env`, `*.pem`, `*.key`, `credentials.json`, and so on) are matched against every tracked file, at any depth.
- **Secret patterns.** Eight regular expressions (GitHub tokens, Chromatic tokens, Nx Cloud tokens, AWS keys, private keys, and so on) are applied to every tracked text file under 1 MB. The transcript is tracked, so it is read this way. It is worth being precise about what this check did and did not do. It catches *shapes* of credential, and it is not what caught the credential that was actually on the machine: that was found by a separate search, run by hand, for the exact values of the credentials held in the local files — a check a gate cannot perform, because a gate does not know what random string on a machine happens to be. The distinction is the point. A pattern check is cheap, mechanical, and covers the shape. A value check is expensive, manual, and covers the instance.
- **Project tags.** Every project must carry a tag from a closed vocabulary. A project with no tags fails, because a project with no tags has no boundary.

5.5 Budgets

Two source budgets, counted separately: 95 files and 3,300 non-blank lines of product source, and 12 files and 400 lines of test source. Product source is 3,073 lines at the end of the study period; the test suite is 227.

The split is deliberate and worth defending, because the alternative is worse. If tests counted against the product budget, the first test would have to cost a line of product code, and the budget would push in the direction of removing verification rather than adding it. Excluding tests without bounding them would make the exclusion a hole. Bounding them separately makes it a rule: the suite can grow to 400 lines and no further without a reviewed decision.

5.6 What the mechanism does not protect

Stating this is part of the mechanism.

It does not make the code good. It does not decide what to build — that was still judgement, and the camera episode shows how a judgement call can be overridden by a fleet. It does not protect against a compromised host (§4.4). It does not catch a wrong change that satisfies every rule. And it does not survive the files it protects being deleted *and* their hashes removed in the same commit, because the manifest that holds the hashes is not itself hash-pinned. What stands in that gap is a `CODEOWNERS` entry naming the owner, which asks rather than requires: the owner is asked to see the change that arrives with an updated hash beside it.

Nor does it stand in for the branch. The ruleset on `main` is what makes the build load-bearing at all — the `Gate` and `Guardrails` checks are required status checks, and no bypass actor can skip them. That is a claim about the repository's configuration rather than about a file inside it, so it is worth saying how a reader checks it: the GitHub API reports the rule, and the appendix

gives the commands. The repository carries two of them for `main`, a branch protection rule and a ruleset, and both require the same two checks. The protection rule sets `strict` and includes administrators; the ruleset requires a pull request but no approving review; neither names a bypass actor. Before the failure the same ruleset required a pull request, no force push and no deletion, and nothing else; the removal satisfied it (§4.1). The mechanism did not replace the ruleset. It gave the ruleset something to require.

---

6. Discussion

6.1 The ratio, and what it means

45 of the 84 commits in the corpus outside the merge group are process work. That ratio drew a criticism worth taking seriously: to a skimming reader, a repository that is 45 parts process to 35 parts product reads as displacement — *couldn't build the thing, built the bureaucracy instead.*

Two observations reconcile it.

First, the ratio is an artefact of the corpus. This repository's product is a PDF annotator; a different product would have a different ratio, and the budgets in §5.5 exist to keep the ratio from becoming the only thing that matters. Second, and more to the point, the process work here *was* the product — of this paper. The decision records, the guardrail script, the manifest and the transcript are not scaffolding around a feature; they are the artefact under study.

The transferable lesson is narrower than "write more process". It is that **the cost of a safeguard is paid in process and should be counted there**. Building the mechanism in §5 took 453 lines of tooling, 17 decision records, and 29 of the corpus's 104 commits — the ones that touched the rules file, the hooks, the guardrail tooling, the budget script or the decision records. That is a real cost, it is paid up front, and it is the kind of thing that should be visible in a commit history rather than hidden.

6.2 The rulebook, rewritten

The rulebook survived, with three changes. Two move constraints from prose into checks; the third marks the limit of what a check can see.

Scope as recorded removals. "Stay inside the brief" is unparseable, and it arrived too late to matter (§3). What replaced it names the paths: the specification harness was removed because the brief did not ask for it, and the *reason* was written down in the manifest as a recorded removal, printed by the guardrail script on every run, so the next reader finds the reason instead of a gap. This is decision record 0009's principle — an unexplained absence reads as an oversight, a recorded exclusion reads as judgement — with the recording moved out of prose and into the build.

Vendor-first, and only where a rule can see it. The prose rule survives, and decision record 0017 names the tldraw examples the work was measured against. What does not survive is any claim that the build enforces it: the lint configuration constrains module boundaries and file size, and the trunk carries a bespoke crop shape of its own. A rule no check can see is a rule that only ever binds whoever reads it.

The gate's required checks are required. A test target was added to the gate, and the rule that used to forbid it was flipped to require it (§5.3).

6.3 What we would do differently

1. **Pin the harness before it exists.** The safeguard against the harness being removed is a hash of the harness. Writing that hash when the harness is created costs one line; writing it after an agent has deleted the harness costs the harness.
2. **State scope as paths, not as principles.** A named boundary is checkable. "Stay inside the brief" is a sentiment.
3. **Fewer agents on one repository.** Four agents working one task concurrently produced up to a 3,400-line fork of a platform primitive, and 30 files of it reached the trunk. Concurrency is the multiplier on every ambiguity in the rulebook.
4. **Publish the finding, not the corpus.** The transcript is evidence; the finding is the contribution. A reader who wants 9,924 lines of raw session can have them from the repository; a reader who wants the lesson should not have to find it.

---

7. Threats to validity

Single case, no comparison. One repository, one operator, one domain, 30 days. There is no control condition and no effect size. The mechanism in §5 is shown to be present and to fail the specific removals it was designed to catch; it was not tested against a randomised baseline. Nothing here supports a claim that mechanically enforced safeguards improve software outcomes.

Self-report. The operator is also the author, and adjudicated every conflict the paper describes. The decision records are his account. Where a finding rests on the decision record alone — F2's account of *why* the agents built their own camera — we say so in the text.

The account and the artefact disagree about sequence. §4.1 reports that the operator's later account attributes the removal to an agent reading "stay inside the brief", a phrase that appears in no file in the repository before the commit that removed the harness and wrote the rule. We report the disagreement rather than resolving it, because resolving it would mean choosing between an account and an artefact, and the artefact is the one a reader can check. It is the clearest instance of the general limitation in this paper: a decision record is authoritative about intent and unreliable about ordering.

The visible history is incomplete. The public history begins 2026-10-05 at a scaffold commit that already contains the harness. The harness's construction is therefore evidenced by the transcript, the manifest's recorded-removals list and one decision record, rather than by commits. F1's *removal* is fully evidenced; the period before it is not.

The attribution instrument is self-declared. `Co-authored-by` trailers are a convention the author adopted. Recent work shows agents are attributable from their output signatures [4]; we did not attempt that classification, and our counts should be read as a record of claims, not of contributions.

The transcript is machine-written by the class of system under study. It is the most complete source for agent reasoning and the least controlled. We used it only where a commit corroborates.

Survivorship. This is a repository that was published rather than abandoned. Repositories where the mechanism failed badly are, by construction, the ones not written up.

One incident, self-reported. The incident in §4.4 has no independent forensic record in this paper.

---

8. Conclusion

Agents optimise against whatever you make checkable, and prose is not checkable. In a 30-day corpus of agents working concurrently on one repository under a prose-only rulebook, an agent correctly deleted the repository's own safeguards and said so in the commit; a directive to use the platform's primitive was written the day after the work it should have governed; and three checks passed while asserting less than they appeared to. The mechanism that followed moves the constraints into the build: hash-pinned protected files, mechanical invariants, required and forbidden gate targets, and separately bounded budgets.

Three design rules follow, and they are the paper's whole argument.

1. **Pin what must survive.** If the loss of an artefact would be unrecoverable, its hash belongs in the build on the day it is created.
2. **Derive checks from the project graph, not from a list somebody edits.** An enumerated list of what to check is a list that drifts.
3. **A check that cannot fail for the right reason is not a check.** If it passes on malformed input, it is not asserting the property it appears to assert, and the reader of the green build is being misled.

The generalisable claim is narrow and, we think, durable: the cost of running agents against a repository is not the code they write. It is the number of things that were true only in prose, and what happens to them the first time an agent reads them correctly — including the rulebook the agent writes afterwards, to explain what it has just done.

References

[1] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub." *ACM Transactions on Software Engineering and Methodology*, 35(10), 2025. doi:10.1145/3798166

[2] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering." arXiv:2507.15003, 2025.

[3] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. "Investigating Autonomous Agent Contributions in the Wild: Activity Patterns and Code Change over Time." *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026. doi:10.1145/3793302.3793354

[4] "Fingerprinting AI Coding Agents on GitHub." *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026. doi:10.1145/3793302.3793582

[5] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Lingming Zhang, Zhenpeng Chen, and Yiling Lou. "Large Language Model-Based Agents for Software Engineering: A Survey." arXiv:2409.02977, 2024.

[6] Burak Yetiştiren, İpek Özsoy, Miray Ayerdem, and Eray Tüzün. "Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT." arXiv:2304.10778, 2023.

[7] Albert Ziegler, Eirini Kalliamvakou, LI X.ALICE, Andrew Rice, Devon Rifkin, Shawn Simister, and Ganesh Sittambalam. "Measuring GitHub Copilot's Impact on Productivity." *Communications of the ACM*, 67(8):54–63, 2024.

[8] tldraw. "Screenshot Tool" and "Pin Bindings" examples, and the camera options documentation. https://tldraw.dev/examples

[9] Nx. Nx documentation: project graph, affected targets, and enforced module boundaries. https://nx.dev

[10] The corpus repository, including its 17 decision records, the guardrail script and manifest, the budget script, and the session transcript. https://github.com/effinrich/h2-fe-assessment

---

Appendix A — Reproducing the counts

Every number in Table 1 is reproducible from the public repository. The commands below are pinned to the commit that ended the study period rather than to the branch tip, so that later commits to the repository do not change what they report.

```
CORPUS=b200e23                # the trunk at the end of the study period

git rev-list --count $CORPUS                         # 104
git log --format=%s $CORPUS | grep -c '^Merge'    # 20 commits with a Merge subject
git rev-list --count --merges $CORPUS              # 19 of them are merges by parent
git log --format=%s $CORPUS | grep -cE '^(feat|fix|perf|refactor|ci|test)'   # 35
git log --format=%s $CORPUS | grep -cE '^(docs|chore|style|build)'            # 45
git log $CORPUS --format=%B | grep -c '^Co-authored-by:'                     # 126
git log $CORPUS --format=%B | grep '^Co-authored-by:' \
  | sed 's/Co-authored-by: //' | sort | uniq -c | sort -rn
```

Table 2:

```
git show --numstat --format='' 9eeb60a      # the removal
git show --shortstat --format='' 9eeb60a    # 137 files changed, +104 -10,097
```

The timeline of §3 and §4.1 — the scope rules were written by the removal, and the guardrail warning came later:

```
git show -s --format='%(trailers)' 9eeb60a        # Co-authored-by: Cursor
git show -s --format=%B 9eeb60a | tail -3        # "... AGENTS.md now leads with scope rules"
git show 9eeb60a^:AGENTS.md | wc -l               # 166 lines before the removal
git show 9eeb60a:AGENTS.md | wc -l                # 67 lines after it
git ls-tree -r --name-only 9eeb60a^ \
  | grep -cE '(^|/)(AGENTS|CLAUDE)\.md$'           # 14 instruction files before it
git grep -ic 'stay inside the brief' 9eeb60a^ -- .   # no hits: the rule did not exist yet
git show 9eeb60a^:AGENTS.md | sed -n '/Rules that do not bend/,/^## /p'   # the old rules
git show 9eeb60a:AGENTS.md  | sed -n '/Scope rules/,/^## /p'             # the new rules
git log --format='%h %ad' --date=short -S'not scaffolding' $CORPUS -- AGENTS.md
```

The camera episode:

```
git show --numstat --format='' a27f292      # PR #47: 29 files, +365 -1,561
git show --numstat --format='' c7fe8ae      # PR #57: 15 files, +98 -370
git diff --stat 6f6ec34 origin/refactor/camera-components \
  -- libs/web/features/camera/              # empty: the trunk matched the branch
git ls-tree -r --name-only 6f6ec34 -- libs/web/features/camera \
  | grep -cE '\.tsx?$'                      # 30 files on the trunk
git ls-tree -r --name-only origin/refactor/camera-structure \
  -- libs/web/features/camera | grep -cE '\.tsx?$'   # 36 files, the variant that never merged
```

The mechanism, and the state of the repository at the end of the study period, run at that commit:

```
bun run budget        # size budget: 81/95 files, 3073/3300 non-blank lines
                      # test budget: 4/12 files, 227/400 non-blank lines
bun run guardrails    # 9 files, 6 invariants, 17 records, 165 tracked files scanned
```

The commits that carried the mechanism (§6.1):

```
git log --format=%h $CORPUS -- tools/ docs/decisions/ AGENTS.md CLAUDE.md \
  lefthook.yml .cursor/hooks.json .claude/settings.json \
  .github/workflows/guardrails.yml | sort -u | wc -l   # 29 of 104
```

The branch protection and the rulesets. These are properties of the repository's configuration rather than of any file in it, so they are read from the API rather than from the tree. The rulesets endpoint is public; the branch protection endpoint answers only to an authenticated caller.

```
gh api repos/effinrich/h2-fe-assessment/branches/main/protection --jq \
  '{checks: .required_status_checks.contexts, strict: .required_status_checks.strict,
    admins: .enforce_admins.enabled, reviews: .required_reviews,
    restrictions: .restrictions}'
# {"admins":true,"checks":["Gate","Guardrails"],"restrictions":null,
#  "reviews":null,"strict":true}

gh api repos/effinrich/h2-fe-assessment/rulesets --jq \
  '.[] | {name, target, enforcement, bypass_actors}'
# feature-branches  branch  active  null
# main              branch  active  null

ID=$(gh api repos/effinrich/h2-fe-assessment/rulesets --jq \
  '.[] | select(.name=="main") | .id')
gh api repos/effinrich/h2-fe-assessment/rulesets/$ID --jq \
  '[.rules[] | {type, reviews: .parameters.required_approving_review_count,
     checks: .parameters.required_status_checks}]'
# deletion, non_fast_forward, pull_request (reviews 0), creation,
# required_status_checks (Gate, Guardrails)
```